

# STSCI 6520 Homework 2 - Problem 1

K.CHAN Kernel Density Estimation and Bandwidth Tuning

```
set.seed(8888)

# The True Density: Standard Normal
p_true <- function(x) {
  dnorm(x, mean = 0, sd = 1)
}

# The Kernel: Epanechnikov Kernel
#  $k(u) = 3/4 * (1 - u^2)$  for  $|u| \leq 1$ 
k_func <- function(u) {
  val <- 0.75 * (1 - u^2)
  val[abs(u) > 1] <- 0
  return(val)
}

# Leave-One-Out (LOO) Pseudo-Likelihood Function
# Returns the sum of log-likelihoods for a specific bandwidth h
# Input: h (bandwidth), x (data vector)
loo_log_lik <- function(h, x) {
  n <- length(x)

  # Compute matrix of differences  $(x_i - x_j) / h$ 
  # outer(x, x, "-") creates an n x n matrix of differences
  diff_matrix <- outer(x, x, "-") / h

  # Apply kernel function
  k_matrix <- k_func(diff_matrix)

  # The LOO estimator removes the i-th point from its own density estimate.
  # In the matrix, this corresponds to the diagonal (where  $x_i - x_i = 0$ ).
  # We set diagonal to 0 effectively removing the point itself.
  diag(k_matrix) <- 0

  # Compute density estimates for each point i using the n-1 other points
  # Sum over columns (j) for each row (i)
  # Formula:  $(1 / ((n-1) * h)) * \sum_{j \neq i} K(\dots)$ 
  dens_estimates <- rowSums(k_matrix) / ((n - 1) * h)

  # Avoid log(0) by replacing 0 with a very small number (machine epsilon)
  dens_estimates[dens_estimates == 0] <- .Machine$double.eps

  # Sum of log likelihoods
  return(sum(log(dens_estimates)))
}
```

## Part (a): Plot Log Pseudo-Likelihood vs $h$ ( $n = 200$ )

```
n_a <- 200
X_a <- rnorm(n_a)

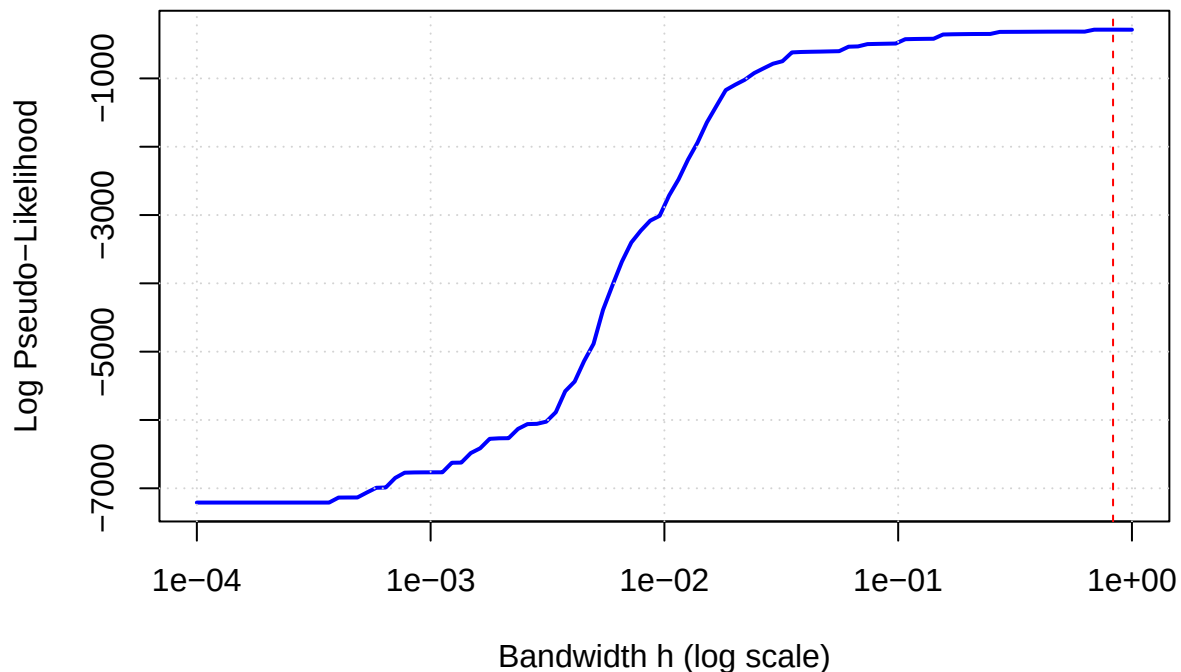
# Prob range is: [10^-4, 1]
# expand the log sequence to plot against likelihood
h_grid <- exp(seq(log(0.0001), log(1), length.out = 100))

# Compute likelihoods for grid
lik_values <- sapply(h_grid, function(h) loo_log_lik(h, X_a))

# Plot
plot(h_grid, lik_values, type = "l", log = "x", lwd = 2, col = "blue",
     main = "A-FINAL SOLUTION: Log Pseudo-Likelihood vs Bandwidth h (n=200)",
     xlab = "Bandwidth h (log scale)", ylab = "Log Pseudo-Likelihood")
grid()

# Find rough max from grid (just for sanity check)
h_best_grid <- h_grid[which.max(lik_values)]
abline(v = h_best_grid, col = "red", lty = 2)
```

## A-FINAL SOLUTION: Log Pseudo-Likelihood vs Bandwidth $h$ ( $n=20$ )



- Approximate optimal  $h$  from grid: 0.8302176 - Log Likelihood at this  $h$ : -256.9273

## Part (b): Optimize h for $n = 200 \dots 2000$ and Plot Density

```
# Generate large dataset for subsets
# This makes the trend in parts (c)/(d) smoother than regenerating fresh data
X_master <- rnorm(2000)
n_seq <- seq(200, 2000, by = 50)
h_opt_values <- numeric(length(n_seq))

# Loop through sample sizes
for (i in seq_along(n_seq)) {
  n_curr <- n_seq[i]
  X_curr <- X_master[1:n_curr]

  # Use R's optimize function to find the precise maximum
  # Search in [0.01, 2] is good for normal dist.
  opt_res <- optimize(f = function(h) loo_log_lik(h, X_curr),
                      interval = c(0.001, 2),
                      maximum = TRUE)

  h_opt_values[i] <- opt_res$maximum
}

# Retrieve optimal h for n=200 (first element)
h_n200 <- h_opt_values[1]
X_200 <- X_master[1:200]

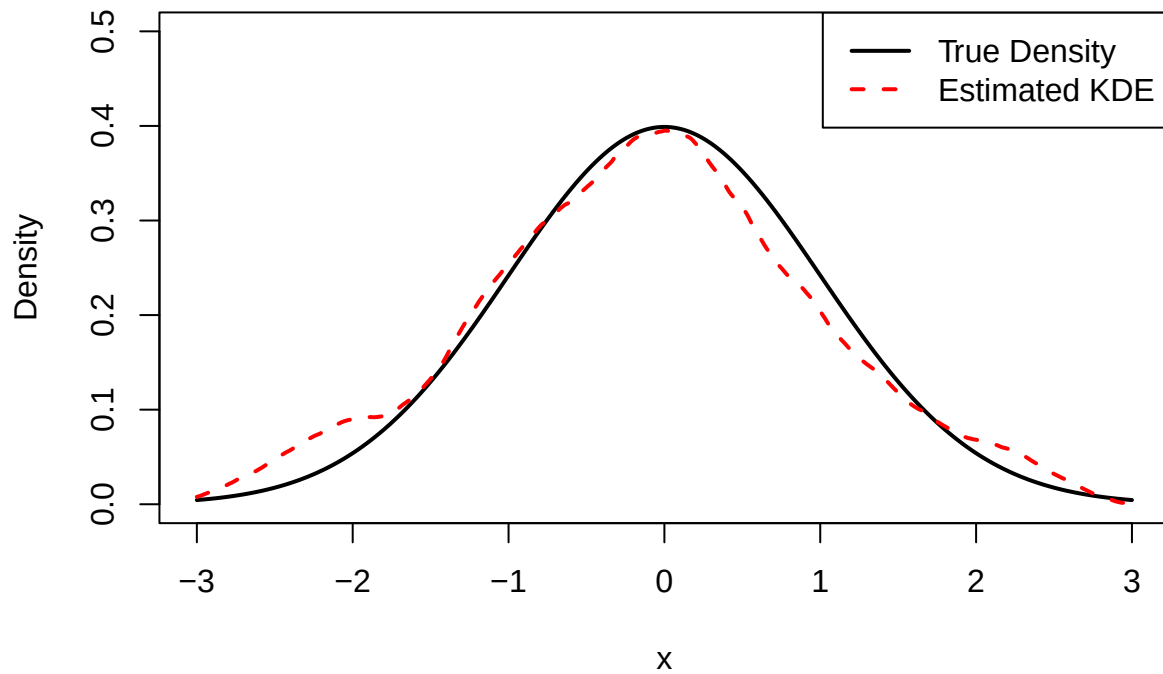
# Plot estimated density vs True density for n=200
x_eval <- seq(-3, 3, length.out = 1000)

# Calculate standard KDE using the optimal bandwidth
kde_est <- sapply(x_eval, function(val) {
  mean(k_func((val - X_200) / h_n200)) / h_n200
})
```

## B FINAL SOLUTION - PLOT KDE”

```
plot(x_eval, p_true(x_eval), type = "l", lwd = 2, col = "black",
     ylim = c(0, 0.5),
     main = paste("KDE vs True Density (n=200, h=", round(h_n200, 3), ")", sep=""),
     xlab = "x", ylab = "Density")
lines(x_eval, kde_est, col = "red", lwd = 2, lty = 2)
legend("topright", legend = c("True Density", "Estimated KDE"),
     col = c("black", "red"), lty = c(1, 2), lwd = 2)
```

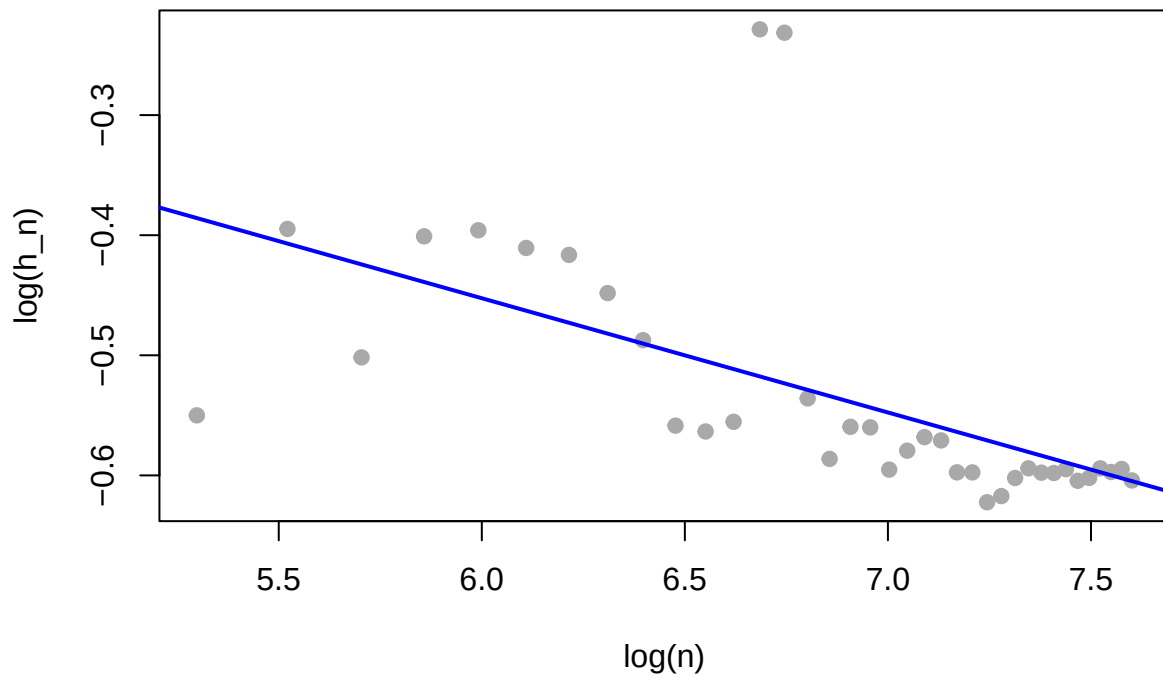
### KDE vs True Density (n=200, h=0.577)



### Part (c): Linear Regression ( $\log h \sim \log n$ )

```
df <- data.frame(  
  log_n = log(n_seq),  
  log_h = log(h_opt_values),  
  log_log_n = log(log(n_seq))  
)  
  
# Plot log-log relation  
plot(df$log_n, df$log_h, pch = 19, col = "darkgrey",  
     main = "Scaling of Optimal Bandwidth with Sample Size",  
     xlab = "log(n)", ylab = "log(h_n)")  
  
# Model 1:  $\log h = \alpha * \log n + \beta$   
model1 <- lm(log_h ~ log_n, data = df)  
abline(model1, col = "blue", lwd = 2)
```

## Scaling of Optimal Bandwidth with Sample Size



C FINAL SOLUTION: Model 1 Coefficients:

```
print(coef(model1))
```

```
## (Intercept)      log_n  
##  0.11824386 -0.09512128
```

Part (d): Complex Regression ( $\log h \sim \log n + \log \log n$ )

```
# Model 2:  $\log h = \alpha * \log n + \gamma * \log(\log n) + \beta$   
model2 <- lm(log_h ~ log_n + log_log_n, data = df)
```

D FINAL SOLUTION: Model 2 Coefficients:

```
print(coef(model2))
```

```
## (Intercept)      log_n  log_log_n  
##   -5.118278   -1.009775    5.990584
```

### Part (e): Compare Sum of Squared Residuals (SSR)

```
ssr1 <- sum(resid(model1)^2)
ssr2 <- sum(resid(model2)^2)
```

Sum of Squared Residuals (Model 1): 0.2372772 Sum of Squared Residuals (Model 2): 0.2082657 ### E:  
FINAL SOLUTION » Model 2 fits the data better (lower SSR).